

Chapter 67: WebSockets

Scenario 1: Wrapping the Native WebSocket API in an Injectable Service

Situation: Your team is building a live trading dashboard. Several unrelated components need to send and receive messages over a single WebSocket connection, but right now every component that needs real-time data creates its own `new WebSocket(url)` instance directly in `ngOnInit`, leading to a dozen open connections to the same endpoint and no central place to handle errors or reconnection.

Question: How would you design an Angular service that wraps the native WebSocket API so components consume it reactively via RxJS, with a single shared connection?

Answer: The core idea is to create an injectable, `providedIn: 'root'` service that owns exactly one `WebSocket` instance and exposes its inbound messages as a multicast `Observable` (so multiple subscribers share the same underlying socket), plus a method to send messages out. Use `Subject / ReplaySubject` internally to bridge the WebSocket's callback-based API (`onmessage`, `onopen`, `onclose`, `onerror`) into RxJS streams, and share the observable with `share()` or a manually managed `RefCount`-style connection so the socket is only opened once regardless of how many components subscribe.

```
import { Injectable, inject, DestroyRef } from '@angular/core';
import { Observable, Subject, share } from 'rxjs';

export interface SocketMessage<T = unknown> {
  channel: string;
  payload: T;
}

@Injectable({ providedIn: 'root' })
export class WebSocketService {
  private socket?: WebSocket;
  private readonly incoming$ = new Subject<SocketMessage>();
  private readonly status$ = new Subject<'open' | 'closed' | 'error'>();

  /** Shared, multicast stream - opening the socket lazily on first subscribe. */
  readonly messages$: Observable<SocketMessage> = this.incoming$.asObservable().pipe(
    share({ resetOnRefCountZero: false }) // keep buffer alive even if all unsubscribe momentarily
  );

  connect(url: string): void {
    if (this.socket && this.socket.readyState === WebSocket.OPEN) return;

    this.socket = new WebSocket(url);

    this.socket.onopen = () => this.status$.next('open');
    this.socket.onclose = () => this.status$.next('closed');
    this.socket.onerror = () => this.status$.next('error');

    this.socket.onmessage = (event: MessageEvent) => {
```

```

    try {
      const parsed: SocketMessage = JSON.parse(event.data);
      this.incoming$.next(parsed);
    } catch {
      console.error('Malformed websocket payload', event.data);
    }
  };
}

send<T>(message: SocketMessage<T>): void {
  if (this.socket?.readyState === WebSocket.OPEN) {
    this.socket.send(JSON.stringify(message));
  } else {
    console.warn('Socket not open - message dropped', message);
  }
}

disconnect(): void {
  this.socket?.close();
  this.socket = undefined;
}

get connectionStatus$(): Observable<'open' | 'closed' | 'error'> {
  return this.status$.asObservable();
}
}

```

A consuming component then just injects the service and filters `messages$` by channel, never touching the raw socket:

```

@Component({ /* ... standalone ... */ })
export class PriceTickerComponent {
  private ws = inject(WebSocketService);
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.ws.connect('wss://api.example.com/stream');
    this.ws.messages$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(msg => { /* handle */ });
  }
}

```

Key tradeoffs: putting `connect()` calls behind idempotency checks avoids duplicate sockets; using `share({ resetOnRefCountZero: false })` prevents tearing down the underlying subject when the last component unsubscribes (e.g., during a route transition) so late resubscribers don't miss a beat. The service, not the component, owns the socket's lifecycle — components only own their subscription lifecycle via `takeUntilDestroyed`.

Interviewer intent: Tests whether the candidate understands the Angular DI singleton pattern for shared stateful resources, and RxJS multicasting to avoid N sockets for N subscribers.

Scenario 2: Automatic Reconnection with Exponential Backoff

Situation: In production, the trading dashboard's WebSocket connection drops whenever the user's laptop sleeps or Wi-Fi flickers. Currently, when the socket closes, nothing happens — the app just goes silent until a manual page refresh. The client wants automatic reconnection but is worried about a "thundering herd" hammering the server if it goes down.

Question: How do you implement reconnection with exponential backoff and a maximum retry ceiling, and how do you notify the UI of connection state during this process?

Answer: Wrap the raw connection attempt in a function that returns an Observable, and use RxJS's `retry` with a configurable `delay` function (or a hand-rolled recursive `timer`-based backoff) rather than a fixed interval. Exponential backoff with jitter avoids synchronized reconnection storms across many clients. Expose a `connectionState` signal so the UI can show "Reconnecting... (attempt 3)" banners.

```
import { Injectable, signal } from '@angular/core';
import { Observable, Subject, retry, timer, throwError } from 'rxjs';
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';

export type ConnState = 'connecting' | 'open' | 'reconnecting' | 'closed';

@Injectable({ providedIn: 'root' })
export class ReconnectingSocketService {
  private socket$: WebSocketSubject<unknown>;
  private readonly maxRetries = 8;
  readonly state = signal<ConnState>('closed');
  readonly attempt = signal(0);

  connect(url: string): Observable<unknown> {
    this.state.set('connecting');

    this.socket$ = websocket({
      url,
      openObserver: { next: () => { this.state.set('open'); this.attempt.set(0); } },
      closeObserver: { next: () => this.state.set('reconnecting') },
    });

    return this.socket$.pipe(
      retry({
        count: this.maxRetries,
        delay: (error, retryCount) => {
          if (retryCount > this.maxRetries) {
            this.state.set('closed');
            return throwError(() => error);
          }
          this.attempt.set(retryCount);
          const backoffMs = Math.min(30_000, 1000 * 2 ** retryCount);
          const jitter = Math.random() * 300;
          this.state.set('reconnecting');
          return timer(backoffMs + jitter);
        },
      })
    );
  }
}
```

```

    }},
  );
}

send(message: unknown): void {
  this.socket$?.next(message);
}
}

```

Design notes:

- **Cap the backoff** (e.g., at 30s) so long outages don't lead to hour-long waits.
- **Add jitter** so hundreds of clients reconnecting after a server restart don't all hit it at the exact same millisecond.
- **Distinguish** a clean `close()` (user navigated away, intentional) from an abnormal closure (`event.wasClean === false` or non-1000 code) — only retry on abnormal closures.
- Surface `attempt` and `state` as signals so templates can reactively show reconnection UI without manual subscription management.

Interviewer intent: Evaluates practical resilience-engineering knowledge — not just "can you reconnect" but whether the candidate thinks about backoff caps, jitter, and avoiding retry storms.

Scenario 3: Multiplexing Multiple Logical Channels Over One Socket

Situation: The app has a chat panel, a live-notifications badge, and a presence indicator ("3 users online"), all of which need real-time updates. Opening three separate WebSocket connections works but the backend team says they'd rather multiplex all three over a single physical connection using a `channel` field in each message envelope.

Question: How do you design a multiplexing layer so each feature module can subscribe to just "its" channel without knowing about the others?

Answer: Use RxJS's `websocket()` multiplex operator (or a hand-built equivalent) which lets you define per-channel subscribe/unsubscribe messages and a filter predicate. Each logical "channel" gets its own Observable, but internally they all share one underlying `websocketSubject`.

```

import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { websocket, websocketSubject } from 'rxjs/websocket';

interface Envelope<T = unknown> {
  channel: string;
  type: 'subscribe' | 'unsubscribe' | 'message';
  payload?: T;
}

@Injectable({ providedIn: 'root' })
export class MultiplexedSocketService {

```

```

private socket$: WebSocketSubject<Envelope> = websocket<Envelope>
('wss://api.example.com/ws');

/** Returns an Observable scoped to one logical channel, sharing the one physical socket.
 */
channel<T>(name: string): Observable<T> {
  return this.socket$.multiplex(
    () => ({ channel: name, type: 'subscribe' }), // sent once, on subscribe
    () => ({ channel: name, type: 'unsubscribe' }), // sent once, on unsubscribe
    (msg) => msg.channel === name, // filters shared stream
  ) as Observable<T>;
}

sendTo<T>(name: string, payload: T): void {
  this.socket$.next({ channel: name, type: 'message', payload });
}
}

```

Consumers stay decoupled:

```

// Chat feature
this.socketService.channel<ChatMessage>('chat').subscribe(msg => this.appendMessage(msg));

// Presence feature – completely unaware of chat or notifications
this.socketService.channel<PresenceEvent>('presence').subscribe(evt =>
this.updateRoster(evt));

```

The server side must cooperate: it needs to track per-connection channel subscriptions and route/filter messages accordingly rather than broadcasting everything to everyone. The win is fewer TCP/TLS handshakes, one auth handshake instead of three, and simpler infra (one connection to load-balance, monitor, and rate-limit). The cost is added complexity in message routing on both ends and the fact that one "noisy" channel can't be independently rate-limited at the transport level — you must do it in application code.

Interviewer intent: Tests understanding of `multiplex()` semantics and channel-based application-layer protocol design over a single transport connection.

Scenario 4: Authenticating a WebSocket Connection — Handshake Token vs First Message

Situation: Security review flagged that your WebSocket connects to `wss://api.example.com/ws?token=<JWT>` — the JWT is visible in server access logs and browser dev tools history because it's a query parameter. The security team wants the token out of the URL.

Question: What are the options for authenticating a WebSocket connection, and what are the tradeoffs between passing the token during the handshake (URL, header, or cookie) versus sending it as the first application message after the connection opens?

Answer: There are three broad strategies:

1. **Query parameter** (`wss://.../ws?token=...`) — simplest, but leaks into server access logs, proxy logs, and browser history. Avoid for anything sensitive.

2. **Custom header during the HTTP upgrade handshake** — the WebSocket API in browsers does **not** allow setting arbitrary headers (unlike `fetch`), so this only works if you control a non-browser client, or you use the `Sec-WebSocket-Protocol` subprotocol field as a smuggling channel for a short-lived token (a known workaround, but hacky and has size limits).
3. **HttpOnly cookie** — if the WebSocket endpoint is same-origin (or CORS-configured appropriately), the browser automatically attaches cookies during the upgrade request. This is the most secure option against XSS-stolen tokens, but is vulnerable to CSRF (a malicious page could open a WebSocket to your endpoint from the victim's browser) unless you validate `Origin` server-side.
4. **First-message authentication** — connect without credentials, then immediately send an `{ type: 'auth', token }` message as the very first frame; the server holds the connection in an "unauthenticated" state and closes it if auth doesn't arrive within N seconds or fails validation.

```
@Injectable({ providedIn: 'root' })
export class AuthenticatedSocketService {
  private socket$?: WebSocketSubject<unknown>;

  connect(url: string, getToken: () => Promise<string>): Observable<unknown> {
    this.socket$ = websocket({
      url,
      openObserver: {
        next: async () => {
          const token = await getToken();
          // First message on the wire IS the auth handshake, not part of the URL.
          this.socket$!.next({ type: 'auth', token });
        },
      },
    });
    return this.socket$.asObservable();
  }
}
```

Tradeoffs: the first-message pattern keeps the token out of URLs/logs and works uniformly across browsers and non-browser clients, but there's a brief window where the connection is "open" yet unauthenticated (mitigated by a server-side timeout that closes unauthenticated sockets, e.g., after 5 seconds) and the server must be careful not to process any other message type before auth succeeds. Cookie-based auth is simpler to implement client-side (nothing extra to code) but requires CSRF mitigation via `Origin` header validation since browsers send cookies on cross-site WebSocket handshakes by default. In practice many production systems combine short-lived, single-use handshake tokens (fetched via a prior authenticated REST call, valid for ~30 seconds, one-time use) passed in the URL — this limits query-param leakage risk since the token is useless after first use or shortly after issuance.

Interviewer intent: Probes real-world security understanding beyond "just add a token" — CSRF, XSS, log leakage, and the browser API's actual constraints (no custom headers on WebSocket upgrade).

Scenario 5: Heartbeat / Ping-Pong to Detect Dead (Half-Open) Connections

Situation: Users report that after their laptop wakes from sleep, the "Live" indicator in the app still shows green, but no messages are ever received again — the app thinks it's connected, but the TCP connection is actually dead (a "half-open" connection where the OS hasn't yet noticed the peer is unreachable).

Question: How do you detect a dead WebSocket connection that hasn't fired a `close` event, and recover from it?

Answer: TCP doesn't reliably notify you when the other end vanishes without a graceful close (e.g., network cable pulled, machine hibernated) — the socket can sit in a zombie "open" state indefinitely from the client's point of view. The fix is an **application-level heartbeat**: periodically send a `ping` and expect a `pong` within a timeout; if it doesn't arrive, treat the connection as dead and force a reconnect.

```
import { Injectable, DestroyRef, inject } from '@angular/core';
import { Subject, timer, Subscription } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class HeartbeatSocketService {
  private socket?: WebSocket;
  private pongTimeout?: Subscription;
  private readonly PING_INTERVAL = 15_000;
  private readonly PONG_TIMEOUT = 5_000;
  private readonly reconnectSignal$ = new Subject<void>();

  connect(url: string): void {
    this.socket = new WebSocket(url);

    this.socket.onopen = () => this.startHeartbeat();
    this.socket.onclose = () => this.teardownHeartbeat();

    this.socket.onmessage = (evt) => {
      const data = JSON.parse(evt.data);
      if (data.type === 'pong') {
        this.pongTimeout?.unsubscribe(); // pong arrived in time, cancel the death timer
        return;
      }
      // ... handle regular messages
    };
  }

  private heartbeatLoop?: Subscription;

  private startHeartbeat(): void {
    this.heartbeatLoop = timer(this.PING_INTERVAL, this.PING_INTERVAL).subscribe(() => {
      if (this.socket?.readyState !== WebSocket.OPEN) return;

      this.socket.send(JSON.stringify({ type: 'ping' }));

      // If no pong arrives within PONG_TIMEOUT, assume the connection is dead.
      this.pongTimeout = timer(this.PONG_TIMEOUT).subscribe(() => {
```

```

        console.warn('Heartbeat timeout – forcing reconnect');
        this.socket?.close(); // triggers onClose -> external reconnection logic
        this.reconnectSignal$.next();
    });
});
}

private teardownHeartbeat(): void {
    this.heartbeatLoop?.unsubscribe();
    this.pongTimeout?.unsubscribe();
}
}

```

The server side must symmetrically respond to `ping` with `pong` (or you can rely on the built-in WebSocket protocol-level ping/pong frames if your server framework exposes them — Node's `ws` library supports `socket.ping()` / `'pong'` events natively at the protocol level, which is more efficient than JSON application messages since it avoids parsing overhead and works even if the app-level message loop is stalled). Using protocol-level pings is preferable when available; application-level heartbeat messages are a fallback when you don't control frame-level behavior (e.g., going through certain proxies that strip control frames) or need heartbeat logic visible in application logs.

Interviewer intent: Tests whether the candidate understands the difference between "the JS `close` event fired" and "the connection is actually alive," a subtlety many miss until it bites them in production.

Scenario 6: Guaranteeing Message Ordering Across Reconnects

Situation: A collaborative document editor sends incremental "operation" messages over a WebSocket. During a brief network blip, the connection drops and reconnects within 2 seconds. QA finds that after reconnecting, two operations arrived out of order, corrupting the document state.

Question: WebSockets guarantee in-order delivery *within a single connection* — so why did messages arrive out of order, and how do you guarantee correct ordering across reconnects and retries?

Answer: WebSocket/TCP guarantees FIFO ordering only for frames sent on the *same* underlying connection. The moment you reconnect, you have a brand-new connection with no ordering relationship to the old one. If the client had queued messages to retry-send after reconnecting, and also had some in-flight messages that were sent right as the connection died (whose delivery status is unknown — did the server receive it before the drop, or not?), naive resend logic can duplicate or reorder operations.

The fix has two parts:

1. **Client-side monotonic sequence numbers.** Every outgoing message gets a strictly increasing `seq` number from a single counter that persists across reconnects. The server acknowledges the highest `seq` it has processed; on reconnect, the client resends everything after the last acknowledged `seq`.
2. **Server-side idempotency / reordering buffer.** The server tracks the last applied `seq` per client and either drops duplicates or buffers out-of-order arrivals until the gap is filled (similar to TCP itself, but at the application level, because a reconnect is effectively a new "TCP" from the transport's perspective).

```
@Injectable({ providedIn: 'root' })
```



```

export class OrderedOpSocketService {
  private seq = 0;
  private lastAcked = 0;
  private readonly pending = new Map<number, unknown>(); // seq -> message, awaiting ack
  private socket$?: WebSocketSubject<any>;

  sendOp(op: unknown): void {
    const seq = ++this.seq;
    const envelope = { seq, op };
    this.pending.set(seq, envelope);
    this.socket$?.next(envelope);
  }

  private onAck(ackedSeq: number): void {
    this.lastAcked = Math.max(this.lastAcked, ackedSeq);
    for (const seq of this.pending.keys()) {
      if (seq <= ackedSeq) this.pending.delete(seq);
    }
  }

  private onReconnect(): void {
    // Replay everything the server hasn't acknowledged, IN ORDER.
    const outstanding = [...this.pending.entries()].sort(([a], [b]) => a - b);
    for (const [, envelope] of outstanding) {
      this.socket$?.next(envelope);
    }
  }
}

```

This is essentially a lightweight, application-level version of TCP's own sequence-number/ack mechanism, necessary because reconnection breaks the transport-level ordering guarantee. For less critical data (e.g., a live "price ticker" where only the latest value matters), you can sidestep this entirely by making messages idempotent/last-write-wins with a server timestamp, so ordering doesn't matter — always render the message with the highest timestamp. Choose the strategy based on whether your data is a *stream of independent facts* (last-write-wins is fine) or a *sequence of dependent operations* (true ordering/ack tracking is required, as in this OT/CRDT-style editor).

Interviewer intent: Distinguishes candidates who know the transport-level guarantee ("ordered within a connection") from those who understand it breaks down exactly at the reconnection boundary — a classic real-world gotcha.

Scenario 7: WebSocket vs Server-Sent Events (SSE) vs Long Polling

Situation: The product team wants "live" order status updates: server pushes updates to the client, but the client never needs to send anything back except an initial subscribe request. A junior developer proposes WebSockets by default "because it's more modern." The tech lead pushes back and asks for a proper comparison before committing.

Question: For a one-directional (server → client) live-update feature, how do WebSockets, Server-Sent Events, and long polling compare, and which would you recommend?

Answer:

Aspect	WebSocket	SSE (EventSource)	Long Polling
Direction	Full duplex	Server → client only (client sends via separate HTTP request if needed)	Client-initiated request/response, repeated
Protocol	Own framing over TCP after HTTP Upgrade	Plain HTTP, <code>text/event-stream</code>	Plain HTTP
Browser reconnection	Manual (you write it)	Built-in automatic reconnect with <code>Last-Event-ID</code> resume	Manual (re-issue request)
Proxy/firewall friendliness	Can be blocked by strict corporate proxies that don't support Upgrade	Just HTTP — passes through virtually everything	Just HTTP — passes through everything
HTTP/2 multiplexing	Works, but WS still needs Upgrade per connection	Plays well, share connection with other requests	Works
Server complexity	Requires stateful connection management, often a dedicated gateway	Simple — just keep a response stream open	Simplest — stateless per request
Overhead	Low per-message overhead once connected	Slightly higher (HTTP headers absent after initial handshake — SSE frames are lightweight text)	High — full HTTP request/response cycle per poll
Binary data	Yes (ArrayBuffer/Blob)	No — text only	Yes (regular HTTP body)

For this specific scenario — server pushes status, client never talks back — **SSE is the better default:**

`EventSource` gives you automatic reconnection with resumption via `Last-Event-ID` for free (no reconnect/backoff code to write and maintain), it's just HTTP so it survives corporate proxies and CDNs that may choke on WebSocket upgrades, and it's simpler on the server (no need for a stateful bidirectional protocol). WebSockets earn their complexity when you need true bidirectional low-latency exchange (chat, collaborative editing, gaming, trading order entry) or binary payloads.

```
@Injectable({ providedIn: 'root' })
export class OrderStatusSseService {
  private readonly zone = inject(NgZone);

  streamOrderStatus(orderId: string): Observable<OrderStatus> {
    return new observable<OrderStatus>((subscriber) => {
      const es = new EventSource(`/api/orders/${orderId}/stream`);
```

```

    es.onmessage = (evt) => this.zone.run(() => subscriber.next(JSON.parse(evt.data)));
    es.onerror = (err) => this.zone.run(() => subscriber.error(err)); // EventSource auto-
    retries before erroring out fully

    return () => es.close();
  });
}
}

```

Long polling is the fallback of last resort — reserved for environments where neither WebSocket nor SSE is available (very old browsers, or middleboxes that break streaming responses); it's simple but wasteful, with meaningful latency (bounded by poll interval) and server load (each poll is a full HTTP round trip with headers).

Interviewer intent: Tests whether the candidate defaults to the trendiest tool or actually reasons about directionality, infra constraints, and built-in reconnection semantics before choosing a transport.

Scenario 8: Testing a Component That Depends on a WebSocket Service

Situation: A component displays a live "connection status" badge and a scrolling feed of incoming chat messages, both driven by `WebSocketService`. The QA lead wants unit tests that don't open real network connections (CI runners have no network access to the staging WebSocket server) and run fast and deterministically.

Question: How do you unit test Angular components/services that depend on WebSockets, without needing a real server?

Answer: Never let a unit test open a real socket. Instead:

1. **Abstract the socket behind an injectable service interface** (as in Scenario 1) so components depend on `WebSocketService`, not `window.WebSocket` directly.
2. In tests, **provide a fake/mock implementation** using Angular's `TestBed` DI overrides, backed by plain RxJS `Subject`s you control manually — push fake messages and assert the component reacts correctly, with zero real I/O.
3. For testing the *service itself* (the reconnection/backoff/heartbeat logic), use a **mock WebSocket class** that intercepts `new WebSocket(url)` and lets the test simulate `onopen`, `onmessage`, `onclose`, `onerror` synchronously, combined with `fakeAsync / tick()` to fast-forward through backoff delays without real waiting.

```

// Testing a component that depends on the service – mock the service, not the socket.
describe('ChatFeedComponent', () => {
  let fixture: ComponentFixture<ChatFeedComponent>;
  let fakeIncoming$: Subject<SocketMessage>;

  beforeEach(() => {
    fakeIncoming$ = new Subject();
    const wsServiceStub = {
      messages$: fakeIncoming$.asObservable(),
      connect: jasmine.createSpy('connect'),
      send: jasmine.createSpy('send'),
    };
  });
});

```

```

};

 TestBed.configureTestingModule({
  imports: [ChatFeedComponent], // standalone
  providers: [{ provide: WebSocketService, useValue: wsServiceStub }],
});

 fixture = TestBed.createComponent(ChatFeedComponent);
 fixture.detectChanges();
});

it('renders an incoming chat message pushed through the fake stream', () => {
  fakeIncoming$.next({ channel: 'chat', payload: { text: 'hello' } });
  fixture.detectChanges();

  const rendered = fixture.nativeElement.textContent;
  expect(rendered).toContain('hello');
});
});

// Testing the reconnection logic itself – mock the global WebSocket constructor.
class MockWebSocket {
  static instances: MockWebSocket[] = [];
  onopen?: () => void;
  onclose?: (ev: { wasClean: boolean }) => void;
  onmessage?: (ev: { data: string }) => void;
  sent: string[] = [];

  constructor(public url: string) { MockWebSocket.instances.push(this); }
  send(data: string) { this.sent.push(data); }
  close() { this.onclose?.({ wasClean: true }); }
  simulateOpen() { this.onopen?.(); }
  simulateAbnormalClose() { this.onclose?.({ wasClean: false }); }
}

describe('ReconnectingSocketService backoff', () => {
  beforeEach(() => {
    (window as any).WebSocket = MockWebSocket;
    MockWebSocket.instances = [];
  });

  it('attempts reconnection with increasing delay after an abnormal close', fakeAsync(() => {
    const service = TestBed.inject(ReconnectingSocketService);
    service.connect('wss://fake').subscribe();

    MockWebSocket.instances[0].simulateOpen();
    MockWebSocket.instances[0].simulateAbnormalClose();

    tick(1000); // first backoff step
    expect(MockWebSocket.instances.length).toBe(2); // reconnected

    MockWebSocket.instances[1].simulateAbnormalClose();
  }));
});

```

```
    tick(2000); // second backoff step (exponential)
    expect(MockWebSocket.instances.length).toBe(3);
  }));
});
```

For integration-level confidence beyond unit tests, spin up a lightweight real WebSocket server in-process (e.g., using the `ws` npm package) for a smaller set of end-to-end tests in a Node/Karma-less test runner, or use tools like Playwright against a docker-composed test backend — but keep the bulk of tests fast, synchronous, and network-free by mocking at the DI boundary.

Interviewer intent: Checks whether the candidate designs for testability up front (service abstraction, DI) rather than treating "how do I test this" as an afterthought once WebSocket code is tangled directly into components.

Scenario 9: Backpressure — the Server Sends Faster Than the UI Can Render

Situation: A live log-streaming viewer receives hundreds of log lines per second from a WebSocket during a traffic spike. The component naively appends every message to an array bound to the template, and the browser tab freezes because Angular's change detection re-renders on every single message.

Question: How do you handle backpressure and rendering throughput when a WebSocket delivers messages far faster than the UI can reasonably render them?

Answer: The WebSocket itself has no backpressure mechanism from the browser side (the browser buffers incoming frames faster than you can stop it — you can only control what you *do* with them once received). The fix lives entirely on the client's processing side: buffer and batch using RxJS operators before committing to change detection.

```
import { bufferTime, filter, map } from 'rxjs';

@Component({ /* standalone, OnPush */
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class LogStreamComponent {
  private ws = inject(WebSocketService);
  private destroyRef = inject(DestroyRef);
  readonly visibleLines = signal<LogLine[]>([]);
  private readonly MAX_BUFFER = 500;

  ngOnInit() {
    this.ws.messages$.pipe(
      filter(m => m.channel === 'logs'),
      map(m => m.payload as LogLine),
      bufferTime(200), // collect messages for 200ms windows
      filter(batch => batch.length > 0),
      takeUntilDestroyed(this.destroyRef),
    ).subscribe(batch => {
      this.visibleLines.update(current => {
        const merged = [...current, ...batch];
        // Drop the oldest lines beyond a cap – an unbounded array is a memory leak.
      });
    });
  }
}
```

```

        return merged.length > this.MAX_BUFFER
        ? merged.slice(merged.length - this.MAX_BUFFER)
        : merged;
    });
  });
}
}

```

Key techniques combined:

- `bufferTime(200)` batches many WebSocket messages into one array delivered every 200ms, so the component (and Angular's change detection, especially relevant with `OnPush` + `signal.update()`) runs dozens of times per second instead of hundreds.
- **A hard cap on retained data** (`MAX_BUFFER`) prevents unbounded memory growth — old lines are dropped, similar to a ring buffer.
- `OnPush` + **signals** ensure re-renders happen only when the signal is actually updated, not on every zone-triggered tick.
- For extremely high-volume cases, consider **virtual scrolling** (`@angular/cdk/scrolling`) so the DOM only ever renders the visible rows regardless of how many thousands of log lines are buffered in memory.
- If the *source* can be throttled (server supports it), send a rate-limit/subscribe-with-sampling request upstream so the server itself sends fewer messages — cheaper than receiving-and-discarding on the client.

Interviewer intent: Tests awareness that WebSockets have no built-in flow control and that unthrottled UI updates are a common self-inflicted performance bug, plus familiarity with RxJS batching operators as the fix.

Scenario 10: Zone.js / Change Detection and WebSocket Callbacks

Situation: After migrating part of the app to `provideZonelessChangeDetection()` (zoneless Angular), a component that updates a plain component-class field inside a raw `websocket.onmessage` handler stops updating the UI at all, even though the network tab shows messages arriving correctly.

Question: Why did the UI stop updating, and what's the correct pattern for consuming WebSocket data in a zoneless (or generally, signal-based) Angular application?

Answer: With Zone.js, any async callback — including a raw `websocket.onmessage` — is monkey-patched so Angular knows "something happened, run change detection." Without Zone.js (zoneless mode), Angular has no way of knowing a plain mutable field changed inside an unrelated third-party callback; it can only react to **signal writes** (or `markForCheck` / `ChangeDetectorRef` calls, or async pipe emissions), because those are the mechanisms zoneless Angular explicitly listens to.

The fix: never write to a plain class field expecting it to show up in the template — always route WebSocket data through a `signal`, a `writableSignal.set/update`, or through RxJS + `toSignal()` / `async` pipe, all of which properly notify Angular's reactivity graph regardless of Zone.js being present.

```

@Component({

```

```

changeDetection: ChangeDetectionStrategy.OnPush,
template: `<div>Last price: {{ lastPrice() }}</div>`,
})
export class PriceComponent {
  private ws = inject(WebSocketService);
  private destroyRef = inject(DestroyRef);

  // Correct: signal write triggers change detection in both zoneful and zoneless apps.
  readonly lastPrice = signal<number | null>(null);

  ngOnInit() {
    this.ws.messages$.pipe(
      filter(m => m.channel === 'price'),
      takeUntilDestroyed(this.destroyRef),
    ).subscribe(msg => {
      this.lastPrice.set((msg.payload as { price: number }).price); // triggers re-render
    });
  }
}

// Equivalent using toSignal() directly on the Observable – even more idiomatic:
export class PriceComponentAlt {
  private ws = inject(WebSocketService);
  readonly lastPrice = toSignal(
    this.ws.messages$.pipe(
      filter(m => m.channel === 'price'),
      map(m => (m.payload as { price: number }).price),
    ),
    { initialValue: null },
  );
}

```

The broader lesson: as Angular moves toward zoneless as the default direction, any integration with non-Angular async APIs (WebSocket, `setTimeout`, third-party SDKs, `IntersectionObserver`, etc.) must explicitly funnel state changes through signals (or trigger `ChangeDetectorRef.markForCheck()` as a lower-level escape hatch) instead of relying on Zone.js's implicit monkey-patching to "just work."

Interviewer intent: Tests currency with the latest Angular reactivity model and whether the candidate understands *why* zoneless breaks naive third-party callback integrations, not just that it does.

Scenario 11: Token Refresh Mid-Connection

Situation: Access tokens expire after 15 minutes, but WebSocket connections in this app often stay open for hours (a dashboard left open on a wall-mounted monitor). The backend, after the token expires, silently starts rejecting all further messages on the still-open socket rather than closing it, so the app doesn't even realize authentication has lapsed.

Question: How do you keep a long-lived WebSocket connection properly authenticated as the underlying access token expires and gets refreshed?

Answer: There are two complementary approaches, and production systems typically use both:

1. **Proactive re-authentication before expiry.** Track the token's expiry time (decode the JWT `exp` claim, or track the refresh response's `expires_in`) and schedule sending a fresh `{ type: 'reauth', token }` message on the already-open socket a comfortable margin (e.g., 60 seconds) before expiry — never wait for a rejection.
2. **Reactive handling of an explicit auth-failure message from the server.** The server, on detecting an expired/invalid token, should send a structured `{ type: 'auth_error' }` message (not just start silently dropping messages) so the client can immediately refresh the token and re-authenticate, or close and reconnect if reauth mid-stream isn't supported by the protocol.

```
@Injectable({ providedIn: 'root' })
export class TokenRefreshingSocketService {
  private socket$: WebSocketSubject<any>;
  private reauthTimer?: ReturnType<typeof setTimeout>;

  constructor(private auth: AuthService) {}

  connect(url: string): void {
    this.socket$ = websocket({
      url,
      openObserver: { next: () => this.scheduleReauth() },
    });

    this.socket$.subscribe({
      next: (msg: any) => {
        if (msg.type === 'auth_error') {
          this.handleAuthError();
        }
        // ... normal message handling
      },
    });
  }

  private async scheduleReauth(): Promise<void> {
    const token = await this.auth.getToken();
    this.socket$?.next({ type: 'auth', token });

    const msUntilExpiry = this.auth.getTokenExpiryMs(token);
    const refreshMargin = 60_000;
    clearTimeout(this.reauthTimer);
    this.reauthTimer = setTimeout(async () => {
      const freshToken = await this.auth.refreshToken();
      this.socket$?.next({ type: 'reauth', token: freshToken });
      this.scheduleReauth(); // reschedule for the new token's expiry
    }, Math.max(0, msUntilExpiry - refreshMargin));
  }

  private async handleAuthError(): Promise<void> {
    const freshToken = await this.auth.refreshToken();
    this.socket$?.next({ type: 'reauth', token: freshToken });
  }
}
```


This mirrors how HTTP interceptors refresh tokens before 401s happen where possible, but adapted to a persistent connection: instead of "refresh on 401, retry the request," it's "refresh before expiry, reauth on the same socket," with a reactive fallback if the proactive path is ever missed (clock drift, a suspended laptop that skipped the timer). Critically, the server must actually *validate* incoming reauth messages and either accept them (updating its notion of the connection's identity/permissions) or close the connection cleanly with a specific close code so the client's reconnection logic knows to fetch a truly fresh token rather than retry with the same rejected one.

Interviewer intent: Tests understanding that WebSocket auth isn't a one-time handshake concern — it's an ongoing lifecycle problem for long-lived connections, distinct from typical short-lived HTTP request auth.

Scenario 12: Handling Multiple Tabs Sharing One WebSocket Connection

Situation: Power users often have the same app open in 5+ browser tabs simultaneously (multiple dashboards for different accounts). Each tab opens its own WebSocket connection to the same endpoint, and the backend team reports a huge spike in concurrent connections and wants the client to consolidate.

Question: How can multiple browser tabs from the same origin share a single WebSocket connection instead of each opening their own?

Answer: The standard approach is to elect one tab as the "leader" that owns the actual WebSocket connection, and have all other tabs communicate with the leader via `BroadcastChannel` (or a `SharedWorker`, which is the more robust solution since it runs independently of any single tab's lifecycle).

Using a `SharedWorker` (the cleaner solution): move the WebSocket connection logic entirely into a `sharedworker` script. Every tab connects to the same worker instance (the browser guarantees one worker instance per origin+script), and the worker holds the single WebSocket connection, broadcasting incoming messages to all connected tabs (`ports`) and relaying outgoing messages from any tab to the server.

```
// shared-socket.worker.ts (runs once per origin, regardless of tab count)
const ports: MessagePort[] = [];
let socket: WebSocket | null = null;

function ensureSocket(url: string) {
  if (socket && socket.readyState === WebSocket.OPEN) return;
  socket = new WebSocket(url);
  socket.onmessage = (evt) => {
    for (const port of ports) port.postMessage({ type: 'message', data: evt.data });
  };
}

self.addEventListener('connect', (e: any) => {
  const port: MessagePort = e.ports[0];
  ports.push(port);

  port.onmessage = (msg) => {
    if (msg.data.type === 'connect') ensureSocket(msg.data.url);
    if (msg.data.type === 'send') socket?.send(msg.data.payload);
  };
});
```

```
port.start();
});
```

```
// Angular service consuming the SharedWorker from any tab
@Injectable({ providedIn: 'root' })
export class SharedWorkerSocketService {
  private worker = new SharedWorker(new URL('./shared-socket.worker', import.meta.url));
  private readonly incoming$ = new Subject<unknown>();

  constructor() {
    this.worker.port.onmessage = (evt) => this.incoming$.next(evt.data.data);
    this.worker.port.start();
  }

  connect(url: string) { this.worker.port.postMessage({ type: 'connect', url }); }
  send(payload: unknown) { this.worker.port.postMessage({ type: 'send', payload }); }
  get messages$() { return this.incoming$.asObservable(); }
}
```

Tradeoffs: `SharedWorker` support is universal in modern evergreen browsers but historically absent in some mobile browsers (notably older mobile Safari) — verify current support for your target audience, and always keep a fallback to a per-tab connection if `SharedWorker` is undefined. A simpler but less robust alternative is `BroadcastChannel` combined with `LocalStorage`-based leader election (a tab claims leadership by writing a timestamped lock key; other tabs listen and relay through the leader; if the leader tab closes, a new one is elected) — more code to get right (races during election, handling the leader tab closing mid-session), but works everywhere `BroadcastChannel` does.

Interviewer intent: Tests whether the candidate is aware of the multi-tab connection multiplication problem in real production apps and knows browser primitives (`SharedWorker`, `BroadcastChannel`) beyond basic RxJS/WebSocket wrapping.

Scenario 13: Sending Binary Data (ArrayBuffer/Blob) Over WebSocket

Situation: A new feature streams live audio-level metering data and periodic compressed telemetry snapshots from an IoT gateway to a monitoring dashboard. JSON-encoding the numeric arrays is wasteful — a snapshot that's 2KB of raw binary balloons to 8KB as a JSON array of numbers, and parsing that JSON on every message causes a visible CPU spike.

Question: How do you send and receive binary data over a WebSocket in Angular, and when is it worth the added complexity over JSON?

Answer: WebSocket natively supports binary frames via `ArrayBuffer` or `Blob`. Set `socket.binaryType = 'arraybuffer'` (default is `'blob'` in browsers) to receive `ArrayBuffer` directly, which is generally more convenient for numeric processing with `TypedArray` views (`Float32Array`, `Int16Array`, etc.) than dealing with `Blob`'s async `.arrayBuffer()` conversion.

```
@Injectable({ providedIn: 'root' })
export class BinaryTelemetrySocketService {
```

```

private socket?: WebSocket;
private readonly telemetry$ = new Subject<Float32Array>();

connect(url: string): void {
  this.socket = new WebSocket(url);
  this.socket.binaryType = 'arraybuffer'; // avoid Blob -> ArrayBuffer async round trip

  this.socket.onmessage = (event: MessageEvent) => {
    if (event.data instanceof ArrayBuffer) {
      // Zero-copy view over the raw bytes – no JSON.parse overhead.
      const samples = new Float32Array(event.data);
      this.telemetry$.next(samples);
    } else {
      // Fallback path for control/JSON messages multiplexed on the same socket.
      const control = JSON.parse(event.data);
      this.handleControlMessage(control);
    }
  };
}

sendCommand(cmd: { type: string; deviceId: string }): void {
  this.socket?.send(JSON.stringify(cmd)); // small control messages remain JSON – clarity over micro-savings
}

private handleControlMessage(msg: unknown) { /* ... */ }

get telemetryStream$(): Observable<Float32Array> {
  return this.telemetry$.asObservable();
}

```

When it's worth it: high-frequency numeric data (audio/video metering, sensor arrays, financial tick data with many fields), where JSON's textual overhead and parse cost are measurable (profiled, not assumed) bottlenecks. A common hybrid pattern — as shown above — multiplexes both binary and JSON frames over the *same* connection: distinguish by checking `event.data instanceof ArrayBuffer` vs a string, using binary only for the genuinely high-volume payloads and keeping JSON for infrequent control/metadata messages where human-readability and debuggability in browser dev tools matter more than the last few bytes of overhead. Avoid premature optimization — introduce binary framing only after profiling shows JSON parsing or payload size is an actual bottleneck, since binary payloads are harder to inspect in dev tools and require both client and server to agree on a strict byte layout (consider a schema like Protocol Buffers or a documented fixed struct layout to avoid drift).

Interviewer intent: Tests awareness that WebSocket isn't JSON-only, and whether the candidate can reason about when the added complexity of binary framing actually pays for itself.

Scenario 14: Graceful Shutdown — Closing Sockets Cleanly on Route Change / App Exit

Situation: Users navigate between a "Live Dashboard" route (which opens a WebSocket) and a "Settings" route (which doesn't need it). Currently the socket from the Dashboard route stays open even after navigating away, and QA also noticed that closing the browser tab sometimes leaves the server thinking the client is still connected for up to 60 seconds (until the server's own idle-timeout fires).

Question: How do you ensure a WebSocket connection is cleanly closed when a component is destroyed or the user closes the tab, and why does it matter to the server?

Answer: Two distinct lifecycle events need handling:

1. **Angular-level cleanup (route navigation, component destruction):** use `DestroyRef.onDestroy()` (or `takeUntilDestroyed()`) to close the socket deterministically when the owning component/service scope is torn down, rather than relying on `ngonDestroy` boilerplate everywhere.
2. **Browser-level cleanup (tab close, refresh, navigation away from the app entirely):** listen for the `beforeunload` / `pagehide` event and proactively call `socket.close(1000, 'client navigating away')` with a clean close code, so the server immediately sees a proper close handshake instead of waiting for a TCP-level timeout to notice the peer vanished.

```
@Injectable({ providedIn: 'root' })
export class DashboardSocketService {
  private socket?: WebSocket;

  constructor() {
    // Best-effort: browsers limit what you can reliably do in pagehide/beforeunload,
    // but a synchronous close() call is safe and commonly honored.
    window.addEventListener('pagehide', () => this.closeCleanly());
  }

  connectForRoute(url: string, destroyRef: DestroyRef): void {
    this.socket = new WebSocket(url);

    // Tied to the *component's* lifecycle, not the whole app's.
    destroyRef.onDestroy(() => this.closeCleanly());
  }

  private closeCleanly(): void {
    if (this.socket && this.socket.readyState === WebSocket.OPEN) {
      this.socket.close(1000, 'Client navigating away'); // 1000 = normal closure
    }
  }
}
```

```

@Component({ /* Dashboard route, standalone */ })
export class DashboardComponent {
  private socketService = inject(DashboardSocketService);
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.socketService.connectForRoute('wss://api.example.com/live', this.destroyRef);
  }
}

```

Why it matters to the server: an open WebSocket typically holds a file descriptor, a slot in a connection pool, possibly a subscription to internal pub/sub topics (e.g., Redis channel subscriptions per client) — all real resources. If clients don't close cleanly, the server relies on its own idle/heartbeat timeout (Scenario 5) to eventually reclaim them, meaning "ghost" connections linger, skewing connection-count metrics, wasting memory, and in pub/sub fan-out systems, wasting CPU broadcasting to sockets nobody is listening to anymore. A close code of `1000` (normal closure) also lets the server distinguish "user left intentionally" from "network dropped" (`1006`) in its logs/metrics, which is valuable for diagnosing real connectivity problems versus normal churn.

Interviewer intent: Tests attentiveness to resource cleanup on both the Angular lifecycle side and the browser/OS side, and awareness that server-side resource accounting depends on clients closing politely.

Scenario 15: Preventing Memory Leaks from Long-Lived Subscriptions

Situation: A performance audit of the app after several hours of continuous use (the wall-mounted dashboard scenario again) shows steadily climbing memory usage. Profiling reveals thousands of retained RxJS `Subscription` objects and detached DOM component instances, even though users navigated away from the live-data view dozens of times over the session.

Question: What WebSocket-related patterns commonly cause memory leaks in long-running Angular apps, and how do you prevent them?

Answer: The most common culprits, all stemming from subscriptions or listeners that outlive the component:

1. **Forgetting to unsubscribe** from `messages$` in a component — every time the component is created and destroyed (e.g., re-entering the dashboard route repeatedly), a new subscription piles onto the shared `subject`, and since the `subject` itself never completes, none of the old subscriptions are ever cleaned up automatically. Always pipe through `takeUntilDestroyed(destroyRef)`.
2. **Closures capturing the wrong scope** — an `onmessage` handler defined inside a component that captures `this` (the component instance) keeps that whole component tree alive in memory even after Angular has destroyed it, if the *service* holding the socket outlives the component and its handler still references it.
3. **Accumulating data structures without bounds** — as in Scenario 9, an ever-growing array of received messages that's never capped will grow indefinitely across a multi-hour session, unrelated to subscription hygiene but equally fatal to memory.
4. **Re-registering window event listeners** (like the `pagehide` listener from Scenario 14) every time a

component initializes, instead of once at the service/app level — each additional listener is retained for the life of the page.

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class LiveDashboardComponent {
  private ws = inject(WebSocketService);
  private destroyRef = inject(DestroyRef);
  readonly latestReadings = signal<Reading[]>([]);

  ngOnInit() {
    this.ws.messages$.pipe(
      filter(m => m.channel === 'readings'),
      takeUntilDestroyed(this.destroyRef), // <-- automatically unsubscribes on component
destroy
    ).subscribe(msg => {
      this.latestReadings.update(list => {
        const next = [...list, msg.payload as Reading];
        return next.length > 200 ? next.slice(-200) : next; // bounded growth
      });
    });
  }
}
```

At the service layer, verify the shared `Subject` pattern itself doesn't leak: if `websocketService` is `providedIn: 'root'`, it lives for the app's lifetime by design (that's intentional and fine), but any *component-scoped* provider or a service created per-route (e.g., provided in a lazy-loaded route's providers array) must explicitly close its socket and complete its subjects in `ngOnDestroy`/`DestroyRef.onDestroy`, or every route visit leaks a socket and a subject.

```
@Injectable() // NOT providedIn: 'root' - scoped to a route/component tree
export class RouteScopedSocketService implements OnDestroy {
  private socket?: WebSocket;
  private readonly incoming$ = new Subject<unknown>();

  connect(url: string) { this.socket = new WebSocket(url); /* ... */ }

  ngOnDestroy(): void {
    this.socket?.close();
    this.incoming$.complete(); // release subscribers, allow GC
  }
}
```

Tools for catching this in practice: Chrome DevTools' Memory tab (heap snapshot diffing across repeated navigation cycles is the single most effective technique — navigate to the dashboard and away 10 times, then diff snapshots and look for detached DOM trees or growing subscription counts), and RxJS-specific lint rules (e.g., `rxjs-no-ignored-subscription`) or Angular ESLint rules that flag `.subscribe()` calls without a corresponding teardown mechanism.

Interviewer intent: Tests systematic memory-leak debugging skill in a realistic long-session scenario, not just "do you know to unsubscribe" in the abstract.

Scenario 16: Rate Limiting Outgoing Messages from the Client

Situation: A collaborative whiteboard app sends a WebSocket message on every `mousemove` event while a user is dragging a shape, resulting in hundreds of messages per second per user and visible lag for other users watching the same board, since the server has to broadcast every single one to every other connected client.

Question: How do you throttle high-frequency outgoing WebSocket messages from user interaction without making the local UI feel laggy?

Answer: The key insight is to decouple **local rendering** (which should stay instant and un-throttled, driven directly by the DOM event) from **network transmission** (which should be throttled/sampled), so the acting user's own experience is smooth while the *broadcast* to others is rate-limited.

```
import { fromEvent, sampleTime, map, takeUntil } from 'rxjs';

@Component({ /* ... */ })
export class WhiteboardComponent {
  private ws = inject(WebSocketService);
  private destroyRef = inject(DestroyRef);
  private elementRef = inject(ElementRef<HTMLElement>);

  ngOnInit() {
    const mouseMove$ = fromEvent<MouseEvent>(this.elementRef.nativeElement, 'mousemove');

    // Local rendering: react to every event, no throttling – stays perfectly smooth for the
    // local user.
    mouseMove$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(evt => {
      this.renderLocalCursor(evt.clientX, evt.clientY);
    });

    // Network transmission: sample at a fixed cadence (e.g., ~20 messages/sec) regardless
    // of event frequency.
    mouseMove$.pipe(
      sampleTime(50), // emits at most once every 50ms, using the LATEST position in that
      // window
      map(evt => ({ x: evt.clientX, y: evt.clientY })),
      takeUntilDestroyed(this.destroyRef),
    ).subscribe(pos => {
      this.ws.send({ channel: 'cursor', payload: pos });
    });
  }

  private renderLocalCursor(x: number, y: number) { /* immediate local canvas update */ }
}
```

Choice of operator matters: `sampleTime(50)` emits the most recent value at fixed intervals (good for continuous position data where only the latest matters and you're fine dropping intermediate ones); `throttleTime(50, undefined, { leading: true, trailing: true })` guarantees both the first and last event of a burst are sent (useful if you don't want to miss the exact final resting position of a drag, since plain `sampleTime` could emit a slightly stale position if the mouse stopped moving between sample ticks — mitigate

by also sending a final unthrottled "drag end" message on `mouseup`); `debounceTime` would be wrong here since it waits for a pause in activity, in this "already lands last mouse position on mouseup" workaround, adding real per-event latency that defeats the purpose of a live cursor.

Sending a final, un-throttled message on the terminating event (`mouseup`) is a common pattern regardless of which throttling operator is chosen, ensuring the final authoritative state is always transmitted even if it falls between sample/throttle windows:

```
fromEvent(document, 'mouseup').pipe(
  takeUntilDestroyed(this.destroyRef),
).subscribe(() => this.ws.send({ channel: 'cursor', payload: this.lastKnownPosition, final: true }));
```

Interviewer intent: Tests RxJS operator fluency for rate-limiting (`sampleTime` vs `throttleTime` vs `debounceTime`) applied to a genuine real-time collaboration problem, plus the "local vs network" decoupling insight.

Scenario 17: Handling Server-Initiated Schema/Version Mismatches

Situation: The backend team ships a breaking change to the WebSocket message envelope format (renaming a field, changing a discriminant). Some users have the app open in a browser tab for days without refreshing, so their client is running old JS that doesn't understand the new message shape, causing silent failures (fields showing as `undefined`) rather than clean errors.

Question: How do you protect a long-lived WebSocket client from silently breaking when the server evolves its message protocol?

Answer: Treat the WebSocket protocol like any other API contract that needs versioning and defensive parsing:

1. **Version every message envelope explicitly** (e.g., `{ v: 2, channel: 'chat', payload: {...} }`), so the client can detect when it receives a version it doesn't understand rather than guessing from missing fields.
2. **Validate incoming payloads at the boundary** using a runtime schema validator (e.g., Zod) rather than trusting `JSON.parse` output blindly and casting with `as`. TypeScript types are erased at runtime and provide zero actual protection against a server sending an unexpected shape.
3. **Force a hard client refresh when a version mismatch is detected**, rather than limping along with corrupted state — this is one of the few legitimate cases for prompting the user to reload.

```
import { z } from 'zod';

const ChatEnvelopeSchema = z.object({
  v: z.literal(2),
  channel: z.literal('chat'),
  payload: z.object({
    text: z.string(),
    authorId: z.string(),
    sentAt: z.number(),
```



```

    }},
  });

@Injectables({ providedIn: 'root' })
export class VersionsSafeSocketService {
  private readonly CLIENT_PROTOCOL_VERSION = 2;
  private readonly incoming$ = new Subject<z.infer<typeof ChatEnvelopeSchema>>();
  private readonly staleClient$ = new Subject<void>();

  handleRawMessage(raw: string): void {
    let parsed: unknown;
    try {
      parsed = JSON.parse(raw);
    } catch {
      console.error('Non-JSON WebSocket payload received', raw);
      return;
    }

    const envelopeVersion = (parsed as { v?: number })?.v;
    if (envelopeVersion !== undefined && envelopeVersion > this.CLIENT_PROTOCOL_VERSION) {
      // Server is speaking a newer protocol than this loaded JS bundle understands.
      this.staleClient$.next();
      return;
    }

    const result = ChatEnvelopeSchema.safeParse(parsed);
    if (!result.success) {
      console.error('Message failed schema validation', result.error, parsed);
      return; // fail closed: drop the message rather than propagate corrupted data
    }

    this.incoming$.next(result.data);
  }

  get staleClientDetected$(): Observable<void> {
    return this.staleClient$.asObservable();
  }
}

```

```

// App-level: prompt for reload rather than silently corrupting UI state.
this.versionsSafeSocket.staleClientDetected$.subscribe(() => {
  this.notificationService.showPersistent(
    'A new version is available. Please refresh to continue receiving live updates.',
    { action: 'Refresh', onAction: () => location.reload() },
  );
});

```

The broader principle: WebSocket connections, unlike a typical page load which always fetches fresh JS, can live far longer than the client bundle's shelf life. Any long-lived-connection system needs an explicit contract-versioning and graceful-degradation story — silently ignoring unrecognized fields feels safe but produces confusing partial-failure bugs (a chat message rendering with a blank author, say) that are much harder to diagnose than a clear "please refresh" prompt.

Interviewer intent: Tests whether the candidate thinks beyond the happy path to API evolution and versioning concerns specific to long-lived connections, and whether they trust runtime validation over compile-time-only TypeScript types for external data.

Scenario 18: Combining WebSocket Data with an HTTP-Fetched Initial Snapshot

Situation: A live order book displays the current state of a stock's bids/asks. On page load, the component needs the *current full snapshot* (fetched via a REST `GET`) and then needs to apply a stream of incremental `diff` messages from the WebSocket to keep it live. A race condition bug was found: sometimes a diff message arrives and is applied *before* the initial snapshot has finished loading, causing an exception, or a diff is missed entirely in the gap between "snapshot fetched" and "WebSocket subscription starts."

Question: How do you correctly sequence an initial HTTP snapshot fetch with a live WebSocket diff stream so you never miss an update or apply a diff before the snapshot exists?

Answer: The classic, race-condition-safe pattern is: **subscribe to the WebSocket diff stream first (and buffer everything it emits), fetch the snapshot second, then replay the buffered diffs that arrived during the fetch, dropping any that are already superseded by the snapshot's own sequence number.**

Doing it in the opposite order (fetch snapshot, then subscribe) risks a gap where diffs sent between the snapshot being generated server-side and the subscription being established are lost forever.

```
import { combineLatest, ReplaySubject, from, take, scan, filter } from 'rxjs';

interface Snapshot { seq: number; bids: PriceLevel[]; asks: PriceLevel[]; }
interface Diff { seq: number; changes: PriceLevel[]; }

@Injectable({ providedIn: 'root' })
export class OrderBookService {
  private ws = inject(WebSocketService);
  private http = inject(HttpClient);

  getOrderBook$(symbol: string): Observable<Snapshot> {
    // 1. Start buffering the live diff stream immediately – before the snapshot fetch even
    // begins.
    const diffBuffer$ = new ReplaySubject<Diff>();
    const diffSub = this.ws.messages$.pipe(
      filter(m => m.channel === `orderbook:${symbol}:diff`),
      map(m => m.payload as Diff),
    ).subscribe(diffBuffer$);

    this.ws.send({ channel: `orderbook:${symbol}`, type: 'subscribe' });

    // 2. THEN fetch the snapshot – any diffs arriving during this fetch are safely queued
    // in diffBuffer$.
    return from(this.http.get<Snapshot>(`/api/orderbook/${symbol}`)).pipe(
      switchMap(snapshot =>
        diffBuffer$.pipe(
          // 3. Drop any buffered diff already reflected in (or older than) the snapshot.
          filter(diff => diff.seq > snapshot.seq),
        )
      )
    );
  }
}
```

```

        scan((book, diff) => this.applyDiff(book, diff), snapshot),
        startWith(snapshot),
    ),
    ),
    finalize(() => diffSub.unsubscribe()),
);
}

private applyDiff(book: Snapshot, diff: Diff): Snapshot {
    // merge diff.changes into book.bids/asks by price level, bump book.seq = diff.seq
    return { ...book, seq: diff.seq, /* ...merged levels */ };
}
}

```

The `ReplaySubject` acting as a buffer is the crucial piece — it guarantees that no diff message emitted between "we asked to subscribe" and "the snapshot HTTP call resolved" is ever lost, because every diff is captured the instant it arrives on the WebSocket, independent of whether anything has "consumed" it yet. The `seq` numbers (assuming the server includes a monotonic sequence number in both the snapshot and every diff) let the client safely discard already-applied or stale diffs rather than double-applying them. This exact pattern (subscribe-then-snapshot, buffer-and-replay, sequence-number reconciliation) is how real exchange market-data feeds and collaborative-editing systems solve the "live stream + point-in-time snapshot" race condition — it comes up constantly in any "diff feed + periodic full state" design.

Interviewer intent: Tests the ability to reason carefully about ordering/race conditions between two independent async sources (HTTP + WebSocket) — a notoriously easy thing to get subtly wrong.

Scenario 19: Debugging "Connection Works Locally but Fails in Production Behind a Load Balancer"

Situation: The WebSocket feature works flawlessly on every developer's machine and in the staging environment (single server instance), but in production — which runs three replicas behind an AWS Application Load Balancer — users report random disconnects roughly every 60 seconds, and sometimes messages sent by User A never reach User B even though both show as "connected."

Question: What are the likely causes of WebSocket instability specifically introduced by a load-balanced, multi-replica production environment, and how would you diagnose and fix them?

Answer: Two classic multi-replica issues, both invisible in a single-instance staging setup:

1. **Load balancer idle timeout shorter than your heartbeat interval.** Most load balancers (including AWS ALB, whose default idle timeout is 60 seconds) will forcibly close a connection with no traffic for longer than the configured idle timeout — even though the app-level connection is otherwise healthy. If your heartbeat interval (Scenario 5) is, say, 90 seconds, the LB kills the connection at 60s before your own ping ever fires. **Fix:** set the heartbeat interval comfortably below the LB's idle timeout (e.g., ping every 30s against a 60s LB timeout), and confirm the LB is actually configured for WebSocket support (sticky enough to not round-robin mid-connection — though HTTP Upgrade connections are inherently pinned to one backend for their duration, this is about idle timeout, not routing).
2. **No shared state across replicas — messages don't fan out.** If User A is connected to replica 1 and User B is connected to replica 2, and replica 1 tries to deliver a message to User B by only looking at its own in-memory list of connected sockets, it will never find User B (who's connected to a different

process entirely). **Fix:** replicas need a shared pub/sub backbone (Redis Pub/Sub, Kafka, NATS, or a managed service like AWS API Gateway WebSockets + SQS/SNS) so that a message received by any replica is broadcast to *all* replicas, each of which then delivers it only to its own locally-connected sockets that are subscribed to that channel/room.

```
// Client-side mitigation while the backend architecture is fixed:
// keep the heartbeat well under any known infra idle-timeout, and treat abnormal
// closes as a signal to reconnect immediately rather than waiting for a long backoff step.
@Injectables({ providedIn: 'root' })
export class LbAwareSocketService {
  private readonly HEARTBEAT_INTERVAL = 25_000; // safely under a 60s ALB idle timeout
  // ... same heartbeat pattern as Scenario 5, tuned to infra reality rather than a round
  number
}
```

Diagnosis steps in practice: (a) check LB access logs / target group configuration for idle timeout settings; (b) correlate disconnect timestamps client-side against that timeout value — a suspiciously exact 60-second cadence is a dead giveaway of an LB timeout rather than an app bug; (c) verify server-side whether the "message never arrived" reports correlate with sender and recipient being on different replica instances (add replica ID to server logs); (d) confirm the pub/sub fan-out layer (Redis etc.) is actually wired up in production configuration and not just present in code but pointed at a misconfigured/unreachable instance.

Interviewer intent: Tests real production/infra debugging experience with WebSockets at scale — a very common "gotcha" that catches teams who've only tested against a single local/staging instance.

Scenario 20: Choosing Between a Single Global Socket Service vs. Per-Feature Sockets

Situation: As the app has grown, five different feature teams have each independently built their own WebSocket connection to five different backend endpoints (chat, notifications, live-collab, presence, admin-monitoring). The browser's per-origin concurrent-connection awareness isn't the bottleneck it used to be with HTTP/1.1, but the architecture review wants to know if this sprawl is actually fine, or if it should be consolidated.

Question: When is it appropriate to have multiple independent WebSocket connections in one app versus consolidating everything into one multiplexed connection (Scenario 3)?

Answer: This is a genuine architectural tradeoff, not a "always consolidate" or "always separate" answer:

Favor separate connections when:

- The features connect to genuinely **different backend services/teams** with independent deploy cycles, auth models, and scaling needs (e.g., chat is owned by one team's microservice, admin-monitoring by another) — coupling them into one multiplexed gateway creates an unwanted organizational and operational dependency.
- Failure isolation matters — if the "admin monitoring" feature's connection has a bug and gets stuck in a reconnect loop, you don't want it able to starve or destabilize the unrelated "chat" connection sharing the same socket.
- Load characteristics differ wildly (a low-frequency presence signal vs. a high-frequency collab-editing

stream) — separate connections make it trivial to apply different heartbeat intervals, backoff policies, or even route through different infrastructure (e.g., collab through a low-latency regional gateway, notifications through a general-purpose one).

Favor one multiplexed connection when:

- All the channels are logically part of the same product surface, owned by the same backend service, and share the same auth session — the chat/notifications/presence example in Scenario 3 is exactly this case.
- Connection overhead is a measured concern (mobile clients on cellular networks pay real latency and battery cost per additional TLS+WebSocket handshake; a corporate proxy environment might rate-limit concurrent connections per client).
- You want a single place to reason about reconnection/backoff/heartbeat logic rather than duplicating (and inevitably making inconsistent) that logic five times across independently-built services.

In practice, most mature systems land on a middle ground: **one multiplexed connection per logically-related backend/team boundary**, not literally one connection for the entire app nor one per micro-feature. Concretely for the scenario given: chat + notifications + presence (likely all served by the same "real-time engagement" backend team) are strong candidates to consolidate into one multiplexed connection using the pattern from Scenario 3, while admin-monitoring (a different team, different audience — admins, not end users, likely different auth scope entirely) and live-collab (which may have fundamentally different latency/ordering requirements, per Scenario 6) are reasonable to keep as separate, independently-tunable connections. The deciding question to ask each team is: *"if this connection's backend is down or misbehaving, should it affect any other feature's real-time behavior?"* — if the answer is no, keep it separate; if the answer is "they're the same system anyway," consolidate.

Interviewer intent: Tests architectural judgment and the ability to articulate a nuanced, tradeoff-based answer rather than reflexively applying "DRY, consolidate everything" or "always isolate everything" as a blanket rule.

Quick Revision Cheat Sheet

- **Wrap, don't scatter:** put the native `WebSocket / WebSocketSubject` behind a single injectable, `providedIn: 'root'` service exposing multicast Observables (`share() / subject`) — never let components instantiate `new WebSocket()` directly.
- **Reconnection needs exponential backoff with a cap and jitter**, not a fixed retry interval — prevents both slow recovery and thundering-herd reconnection storms after server restarts.
- **Multiplex logical channels over one physical socket** with RxJS's `WebSocket().multiplex()` (subscribe/unsubscribe messages + a filter predicate) when features share the same backend and auth session.
- **Never put auth tokens in query strings for anything sensitive** — prefer a first-message auth handshake (with a server-side unauthenticated-connection timeout) or same-origin HttpOnly cookies (with `origin` validation against CSRF); the browser WebSocket API cannot set custom headers on the handshake.
- `close` **events don't catch every dead connection** — implement an application-level (or protocol-level) heartbeat/ping-pong to detect half-open TCP connections after sleep/network blips.
- **Ordering is only guaranteed within one connection** — reconnects break that guarantee; use

monotonic sequence numbers, server-side acks, and replay-from-last-ack for operation-dependent data (last-write-wins with timestamps is fine for independent "latest value" data).

- **Choose the transport by directionality and infra constraints:** SSE for server-to-client-only streams (free auto-reconnect via `EventSource`, plain HTTP, proxy-friendly); WebSocket for true bidirectional/low-latency/binary needs; long polling only as a last-resort fallback.
- **Test by mocking at the DI boundary** — stub `WebSocketService` with plain RxJS Subjects for component tests; mock the global `WebSocket` constructor plus `fakeAsync` / `tick()` to test reconnection/backoff timing deterministically, with zero real network I/O.
- **Guard against backpressure** — `bufferTime`/batching plus a hard cap on retained/rendered data (and virtual scrolling for large lists) prevents high-frequency streams from freezing the UI; WebSocket itself has no client-side flow control.
- **Zoneless Angular requires explicit signal writes** (or `toSignal()` / async pipe) for WebSocket-driven UI updates — a plain field mutation inside a raw `onmessage` callback silently stops updating the view without Zone.js's implicit monkey-patching.
- **Long-lived connections need ongoing auth, not one-time handshake auth** — proactively re-authenticate before token expiry and reactively handle an explicit `auth_error` message from the server.
- **Multiple tabs from the same origin should consolidate connections** via `SharedWorker` (preferred) or `BroadcastChannel` + leader election, to avoid multiplying server-side connection load per user.
- **Clean up deterministically:** `DestroyRef.onDestroy()` / `takeUntilDestroyed` for Angular-lifecycle teardown, plus a `pagehide` / `beforeunload` listener sending a clean `close(1000, ...)` so servers reclaim resources immediately instead of waiting on their own idle timeout.
- **Version your message envelopes and validate at runtime** (e.g., with Zod) — TypeScript types don't protect against a server sending a shape your currently-loaded JS bundle doesn't understand on a page left open for days.
- **Sequence HTTP snapshot + WebSocket diff streams carefully:** subscribe to and buffer the diff stream *before* fetching the snapshot, then replay/reconcile by sequence number — never fetch-then-subscribe, or you risk losing diffs in the gap.
- **Production load balancers introduce failure modes invisible in single-instance staging:** tune heartbeat intervals below the LB's idle timeout, and ensure multi-replica backends fan out messages via shared pub/sub (Redis, etc.) rather than assuming any one server process can see every connected client.
- **Consolidating connections is a tradeoff, not a default** — multiplex channels that share a backend/team/auth boundary; keep connections separate across independent services or wildly different latency/failure-isolation needs.

Created By - Durgesh Singh